

SPACES OS — Reporte de avance y funcionamiento

Estado al 25/06/2026 · Rama `cierre/auditoria-completitud` · Demo local (Web :3000 · Postgres :5433).

Documento de avance: qué hace el sistema, cómo funciona cada parte y qué falta.

Verificado contra el código y la base de datos reales, no asumido.

1. Resumen ejecutivo

SPACES OS es una plataforma de gestión para una operadora de publicidad exterior (OOH/DOOH — espectaculares físicos y pantallas digitales). Cubre el ciclo completo de una operadora: inventario de sitios → comercialización → propuesta → campaña → producción (impresión/creativos) → instalación en campo → **validación de publicación** → reporte probatorio → facturación → cobranza.

- **Funcionalidad de negocio:** ~95% — la cadena comercial-fiscal está completa y conectada de extremo a extremo, con persistencia real en base de datos, autenticación, permisos por rol, bitácora y notificaciones.
- **Listo para producción:** ~55–60% — lo que falta no es funcionalidad sino **infraestructura y habilitaciones externas**: aislamiento multi-tenant real (RLS), timbrado fiscal legal (PAC/CFDI), integraciones reales (AdMobilize/CMS) y despliegue. Todos descritos en §8.

Es un producto **funcionalmente rico y demostrable hoy**, a la espera de decisiones de arquitectura/contratos para los últimos pasos de producción.

2. Arquitectura — cómo está construido

Capa	Tecnología	Rol
Frontend	Next.js 14 (App Router), React 18, Tailwind, Radix UI, MapLibre GL	UI del shell + módulos
Estado cliente	Zustand (<code>store.ts</code>)	Se hidrata desde <code>/api/estado</code> y se refresca tras cada escritura
Backend (BFF)	Route Handlers en <code>apps/web/app/api/**</code>	La API viva: valida permisos, ejecuta reglas de negocio
Base de datos	PostgreSQL (:5433 , vía pool <code>lib/server/db.ts</code>)	Fuente de verdad
Auth	bcrypt + sesiones en BD + cookie httpOnly	Login real, no mock

Patrón de datos (clave para entender el resto):

1. La UI lee todo desde un único endpoint `GET /api/estado` que devuelve "rebanadas" (sitios, campañas, reservas, etc.) y llena el store. 2. Cada acción del usuario llama una función de `lib/data/estado-api.ts` → hace `fetch` a un route handler → el handler aplica permisos (`exigir`) y reglas (repos en `lib/server/*-repo.ts`) → escribe en Postgres. 3. Tras escribir, el cliente vuelve a llamar `refrescarEstado()` → la UI queda al día.

Las **derivaciones** (etapa del pipeline, candado de facturación, márgenes, semáforos) viven en funciones puras (`lib/data/derive.ts`), no en la BD: se calculan siempre igual y sin duplicar lógica.

Nota técnica. Existe un `apps/api` (Fastify) que quedó **huérfano**; el backend real es el BFF de Next. Los tipos del front espejan el schema de Prisma a propósito, para que conectar un backend real sea un cableado 1:1.

3. Cómo funciona — recorrido por módulos

El menú lateral muestra solo lo que cada rol puede ver (no se "desactiva": directamente **no se monta**). En celular el menú es un **drawer retráctil** (botón ☰).

Dashboard (Dueño)

Vista de negocio de un vistazo: KPIs (ingreso contratado, margen, por cobrar, ocupación), desglose del **motor de costos** (espacios + impresión + operación), ocupación por día/semana/mes, alertas (rentas vencidas, contratos por vencer, cobranzas, incidencias), mapa de la red y campañas por finalizar.

Agregar inventario (Dueño)

Alta de sitios uno por uno o por **carga masiva** (Excel/CSV con parser real). Captura especificaciones (medidas, tipo de estructura, caras), precios (tarifa publicada, costo de compra interno para margen), datos DOOH (spots, resolución, CMS) y la opción **Computer Vision** (medición de audiencia / AdMobilize), con imagen de detección en local.

Comercial (Dueño, Comercial)

El corazón de la venta: **mapa + lista de inventario** con filtros (tipo, distrito, disponibilidad, precio). Se seleccionan sitios libres y se **reserva** (crea reserva tentativa). Desde aquí se **confirma** o **extiende** una reserva. Colores unificados (mapa y badges): **azul = digital** · **verde = disponible** · **rojo = ocupado** · **ámbar = reservado**. El nombre del lugar aparece al acercar el zoom y al pasar el mouse sobre el pin.

Creativos (Dueño, Comercial)

Por campaña digital: subir imágenes o código (HTML), **aprobar/rechazar** cada creativo y **asignar** cuál se exhibe en cada spot reservado (y cuántas veces). Una campaña **fija (OOH)** **no recibe creativos** (se valida en servidor, responde 409).

Campañas + Pipeline (Dueño, Comercial) — la pieza estrella

Cada campaña "viaja sola" por la empresa con un **pipeline de 10 etapas** que se **deriva del estado real** (no se teclea). Las etapas dependen del tipo de medio:

- **DOOH (digital)**: recibe y valida creativo, **sin imprenta**.
- **OOH (fija)**: se imprime y monta, **sin etapas de creativo**.
- **Híbrida**: ambas.

En el detalle de campaña: pipeline en vivo, **validación de publicación** (\$4), candado de facturación, comercial (subtotal/IVA/total), **rentabilidad** (margen con el motor de costos), reporte de cumplimiento (contratado vs entregado + testigos), ODC, sitios, imprenta, OT, creatividades y evidencias fotográficas.

Clientes (Dueño, Comercial)

CRUD con datos fiscales (RFC, razón social, régimen, CP, uso CFDI). Se configura el **IVA (%)** y la **comisión de agencia (%)** por cliente, que alimentan propuestas y facturación.

Propuestas (Dueño, Comercial)

Cotización por el **método del divisor**: el precio de lista es bruto y $\text{neto} = \text{bruto} \times (1 - \text{comisión}/100)$. **Aprobación granular** sitio por sitio. Cuando la propuesta está aprobada, un botón **"Generar campaña"** la convierte en campaña + reservas, **solo con los sitios aprobados** y precio neto de comisión (cadena conectada, no dos islas).

Network, Arrendadores, Operaciones, Imprenta, Finanzas

- **Network**: sitios compartidos a la red programática.

- **Arrendadores:** propietarios de los predios, contratos de renta, **registro de pagos** (con fecha) y renovaciones; alta de nuevo propietario.
- **Operaciones:** órdenes de trabajo (montaje/mantenimiento); en la OT el sitio se autoselecciona desde la campaña elegida. El cierre en campo (vista móvil) sube foto-testigo con fecha/geo.
- **Imprenta:** órdenes de impresión + **prueba de color** (probatorio). Rechaza órdenes para campañas DOOH (409).
- **Finanzas:** genera factura (exige RFC + razón social, IVA por cliente, **folio fiscal simulado**) y gestiona **cobranza** (buckets 60/90/120, semáforo, abonos parciales).

Integraciones, Actividad, Administración (Dueño)

- **Integraciones:** stubs detectados por entorno (AdMobilize / CMS / CFDI) — simulados hasta tener llaves.
- **Actividad:** bitácora de quién hizo qué y cuándo.
- **Administración:** configuración del negocio + usuarios/roles.

4. Validación de publicación (lo más reciente)

Nueva etapa de **moderación antes de que una campaña salga al aire**, en el detalle de la campaña:

1. **Enviar al dominio / CMS** — marca la campaña como enviada y deja la validación en **Pendiente**. En medios digitales (DOOH/híbrida) exige al menos un anuncio cargado. 2. **Verificar la información de los anuncios** — el panel muestra cada creativo con miniatura y su estatus (validado/pendiente/rechazado), para revisar el contenido antes de publicar. 3. **Aprobar / Rechazar** —

- **Aprobar** → la campaña pasa a **ACTIVA** (al aire) y queda el sello **aprobada por / fecha**.
- **Rechazar** → se captura el motivo y se baja la bandera de envío para corregir y reenviar.

La lista de Campañas muestra arriba una **cola "Por validar"** con las campañas enviadas que esperan revisión. Todo queda en bitácora y dispara notificaciones.

5. Roles y permisos (RBAC)

Seis roles: **Dueño, Comercial, Operaciones, Imprenta, Finanzas, Cliente externo**. Cada acción de escritura pasa por un guard de servidor (`exigir(módulo, acción)`); el cliente externo solo ve su **portal** (sin módulos internos). El renderizado del menú por rol y el guard de API son independientes (defensa en profundidad).

6. La cadena comercial-fiscal de extremo a extremo

```

Cliente (RFC) → Propuesta (divisor, aprobación granular) → [Generar campaña]
  → Campaña + Reservas (solo sitios aprobados)
  → Creativos / Imprenta (según tipo de medio)
  → Operación en campo (OT + foto-testigo)
  → Enviar al dominio → Validación de publicación → ACTIVA (al aire)
  → Reporte de cumplimiento + Candado (OC + testigos + reporte)
  → Factura (IVA por cliente, folio fiscal simulado)
  → Cobranza (buckets / semáforo / abonos)

```

Esta cadena está **conectada y verificada** con un **smoke test** E2E repetible (`scripts/smoke-e2e.mjs` , datos `TEST_` autolimpiables).

7. Cambios de esta sesión

Cambio	Detalle
Validación de publicación	Enviar al dominio + aprobar/rechazar + cola "Por validar" + bitácora/notificaciones (migración aditiva de 6 columnas en <code>campanas</code>).
Pantallas a ancho completo	Todos los módulos usan todo el ancho; modales y panel lateral más anchos (más visual).
Menú retráctil en celular	Drawer con botón hamburguesa; se cierra al navegar o tocar fuera. Sidebar estático en escritorio.
Ocupado en rojo	El badge de sitio "Ocupado" se unificó a rojo, igual que el pin del mapa.

Cambios previos recientes ya integrados: IVA 16% configurable por cliente + comisión de agencia; badge "Computer Vision On/Sin Computer Vision"; alta de propietario + fecha de pago de renta; recolor de pines del mapa; guard inverso OOH→creatividad; conexión propuesta→campaña; smoke test E2E.

8. Estado de avance y qué falta

Leyenda:  hecho y verificado ·  parcial ·  falta — bloqueado por DECISIÓN ·  falta — bloqueado por EXTERNO ·  requiere autorización humana

Área	Estado
Inventario (CRUD, specs, precios, imagen, carga masiva)	
Comercial (mapa/lista, reservar/confirmar/extender)	
Colisión de fechas / sobre-reserva (estáticas)	
Clientes + datos fiscales + IVA/comisión por cliente	
Propuestas (divisor) + aprobación granular	
Conexión propuesta→campaña	
Campañas + pipeline por tipo de medio	
Máquina de estados (DOOH→Imprenta y OOH→creativo)	
Creativos (alta/aprobación/asignación)	
Validación de publicación de campaña	
Operaciones / OT + foto-testigo	
Imprenta + prueba de color	
Reporte probatorio (contratado vs entregado)	
Candado de facturación (OC + testigos + reporte)	
Finanzas (factura con IVA por cliente + cobranza)	 folio fiscal simulado
Motor de costos + margen real	
Auth (bcrypt/sesiones/cookie) + RBAC	
Notificaciones por evento	

Área	Estado
Persistencia real (UI ↔ BD)	✓
Bitácora / auditoría	● registra todo, pero tabla mutable
Storage de evidencias	● base64 en BD; Spaces listo pero inactivo
Multi-tenant / RLS	● instalada pero inerte (superuser / sin FORCE / sin <code>app.tenant_id</code>)
Integraciones (AdMobilize/CMS/CFDI/SUNAT)	✖ stubs; requieren contratos/keys
Facturación fiscal legal (timbrado PAC)	✖ requiere PAC
Deploy / CI-CD / backups	● fuera de alcance actual

Lo que falta, por tipo de bloqueo

- **Decisión (arquitectura):** activar el aislamiento real entre clientes (RLS con rol restringido por request y `app.tenant_id`); definir despliegue/CI/backups. *Bloqueante #1 de producción.*
- ✖ **Externo (contratos/keys):** timbrado fiscal real (PAC CFDI MX / SUNAT PE) — hoy el folio es simulado; métricas y proof-of-play reales de AdMobilize/CMS.
- 🔒 **Autorización humana:** activar el bucket de Spaces de producción en el BFF (llaves existen, no cableadas) y correr el backfill de evidencias base64→Spaces (script listo, no ejecutado).
- **Cerrable por código (opcional):** inmutabilidad de la bitácora (append-only); etapa visible "Al aire" en el pipeline si se desea.

9. Cómo correrlo en local

- **Base de datos:** Postgres en Docker (`spaces_db` , puerto `:5433`).
- **App:** `npm run dev` en `apps/web` → `http://localhost:3000/spaces-dooh/demo/login` .
- **Datos:** ficticios; moneda MXN; IVA 16%.
- **Smoke test E2E:** `node apps/web/scripts/smoke-e2e.mjs` (cadena completa, autolimpiable).

Demo · datos ficticios. La funcionalidad de negocio está completa y demostrable; los pendientes son de infraestructura (multi-tenant), habilitación fiscal legal e integraciones externas.